

INSTITUT NATIONAL DES SCIENCES DE GESTION

BP : 190 Libreville-Gabon / TEL : 01.73.28.45

Site web : www.insg-gabon.com / Adresse mail : contact@insg-gabon.com



Devoir API REST

Niveau : Master 1 IG

Travaux Pratique : Hospital API

Redigé par :

MOGUANGUE Auguste Arcel

Année académie : 2025-2026

Introduction

Les systèmes d'information hospitaliers constituent l'un des domaines les plus exigeants du développement logiciel, nécessitant fiabilité, disponibilité et scalabilité. Ce travail pratique vise à concevoir une API REST complète pour la gestion d'un établissement de santé, en couvrant la gestion des patients, des médecins et des consultations médicales.

Le projet, intitulé Hospital API, combine deux technologies complémentaires : Spring Boot pour l'exposition des services REST et MongoDB pour la persistance des données. Cette combinaison est particulièrement adaptée aux données médicales, dont la structure peut évoluer (ajout de champs, documents imbriqués pour les antécédents, etc.).

Spring Data MongoDB est le module Spring qui assure l'intégration entre Spring Boot et MongoDB. Il fournit des repositories prêts à l'emploi et supprime la majorité du code boilerplate de persistance.

Les compétences mises en pratique incluent : la configuration Spring Boot avec MongoDB, la modélisation des documents (entités), la création de repositories, l'implémentation de contrôleurs REST avec les quatre méthodes HTTP (GET, POST, PUT, DELETE) et la validation via Postman.

I. Mise en place du projet

Le projet a été généré via Spring Initializr avec les dépendances nécessaires à la combinaison Spring Boot et MongoDB.

Paramètres Spring Initializr

Paramètre	Valeur
Type de projet	Maven
Langage	Java 17
Group ID	com.hospital
Artifact ID	hospitalapi
Dépendance 1	Spring Web
Dépendance 2	Spring Data MongoDB
Dépendance 3	Lombok (optionnel)

Configuration MongoDB — application.properties

La connexion à MongoDB est configurée dans le fichier src/main/resources/application.properties :

```
# Connexion MongoDB
spring.data.mongodb.host=localhost
spring.data.mongodb.port=27017
spring.data.mongodb.database=hospital_db
```

```
# Port du serveur
server.port=8080

# Affichage des requêtes MongoDB en console (dev)
logging.level.org.springframework.data.mongodb=DEBUG
```

MongoDB doit être installé et démarré localement (mongod) ou via MongoDB Atlas (cloud). La base hospital_db est créée automatiquement au premier démarrage de l'application.

II. Modélisation des données

Trois entités principales ont été modélisées. Chaque classe Java est annotée avec `@Document` pour être mappée sur une collection MongoDB. L'annotation `@Id` désigne le champ identifiant (ObjectId généré automatiquement).

2.1 Entité : Patient

Représente un patient hospitalisé ou suivi en consultation. Le champ `antecedents` est un document imbriqué permettant de stocker les informations médicales sans collection supplémentaire.

```
package com.hospital.hospitalapi.model;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "patients")
public class Patient {

    @Id
    private String id;           // ObjectId MongoDB
    private String nom;
    private String prenom;
    private String dateNaissance; // Format : "YYYY-MM-DD"
    private String telephone;
    private String email;
    private String groupe_sanguin; // "A+", "B-", "O+"...
    private Antecedents antecedents; // Document imbriqué

    // Constructeurs, Getters, Setters
}

// Classe imbriquée (Embedding)
public class Antecedents {
    private List<String> maladies; // ["Diabète", "Hypertension"]
    private List<String> allergies; // ["Pénicilline"]
    private List<String> medicaments; // Traitements en cours
}
```

2.2 Entité : Medecin

Représente un médecin de l'établissement. Le champ specialite permet de filtrer les médecins par domaine (cardiologie, pédiatrie, etc.).

```
package com.hospital.hospitalapi.model;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "medecins")
public class Medecin {

    @Id
    private String id;
    private String nom;
    private String prenom;
    private String specialite; // "Cardiologie", "Pédiatrie"...
    private String telephone;
    private String email;
    private String numeroOrdre; // Numéro d'ordre médical
    private boolean disponible; // true/false

    // Constructeurs, Getters, Setters
}
```

2.3 Entité : Consultation

Collection centrale du système. Une consultation lie un patient à un médecin via leurs identifiants. Le diagnostic et l'ordonnance sont directement intégrés (embedding).

```
package com.hospital.hospitalapi.model;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import java.time.LocalDateTime;

@Document(collection = "consultations")
public class Consultation {

    @Id
    private String id;
    private String patientId; // Référence → patients
    private String medecinId; // Référence → medecins
    private LocalDateTime date; // Date et heure
    private String motif; // Raison de la consultation
    private String diagnostic; // Diagnostic posé
    private Ordonnance ordonnance; // Document imbriqué
    private String statut; // "planifiée"|"terminée"|"annulée"

    // Constructeurs, Getters, Setters
}

// Ordonnance imbriquée
public class Ordonnance {
    private List<String> medicaments; // ["Paracétamol 500mg"]
    private String instructions; // Posologie
    private int dureeJours; // Durée du traitement
}
```

Structure de hospital_db : 3 collections (patients, medecins, consultations). Les consultations référencent patients et médecins par ObjectId, et intègrent l'ordonnance par embedding.

III. Repositories Spring Data MongoDB

Spring Data MongoDB génère automatiquement les requêtes à partir du nom des méthodes déclarées dans les interfaces Repository. Aucune implémentation manuelle n'est nécessaire.

PatientRepository

```
package com.hospital.hospitalapi.repository;

import com.hospital.hospitalapi.model.Patient;
import org.springframework.data.mongodb.repository.MongoRepository;
import java.util.List;

public interface PatientRepository
    extends MongoRepository<Patient, String> {

    // Spring génère automatiquement la requête MongoDB
    List<Patient> findByNom(String nom);
    List<Patient> findByGroupeSanguin(String groupe);
    boolean existsByEmail(String email);
}
```

MedecinRepository

```
public interface MedecinRepository
    extends MongoRepository<Medecin, String> {

    List<Medecin> findBySpecialite(String specialite);
    List<Medecin> findByDisponibleTrue();
    boolean existsByNumeroOrdre(String numero);
}
```

ConsultationRepository

```
public interface ConsultationRepository
    extends MongoRepository<Consultation, String> {

    List<Consultation> findByPatientId(String patientId);
    List<Consultation> findByMedecinId(String medecinId);
    List<Consultation> findByStatut(String statut);
    List<Consultation> findByDateBetween(
        LocalDateTime debut, LocalDateTime fin);
}
```

IV. Contrôleurs REST

Trois contrôleurs REST ont été développés, un par entité. Chacun implémente les opérations CRUD complètes (GET, POST, PUT, DELETE) ainsi que des endpoints de recherche spécifiques.

4.1 PatientController — Endpoints

Méthode	URL	Statut	Description
GET	/api/patients	200 OK	Lister tous les patients
GET	/api/patients/{id}	200 / 404	Récupérer un patient par ID
GET	/api/patients/nom/{nom}	200 OK	Rechercher par nom
GET	/api/patients/groupe/{groupe}	200 OK	Filtrer par groupe sanguin
POST	/api/patients	201 Created	Ajouter un patient
PUT	/api/patients/{id}	200 OK	Modifier un patient
DELETE	/api/patients/{id}	204 No Content	Supprimer un patient

4.2 Implémentation — PatientController

```
package com.hospital.hospitalapi.controller;

import com.hospital.hospitalapi.model.Patient;
import com.hospital.hospitalapi.repository.PatientRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/patients")
public class PatientController {

    private final PatientRepository repo;

    public PatientController(PatientRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public List<Patient> getAll() {
        return repo.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Patient> getById(@PathVariable String id) {
        return repo.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/nom/{nom}")
    public List<Patient> getByName(@PathVariable String nom) {
        return repo.findByName(nom);
    }

    @PostMapping
    public ResponseEntity<Patient> create(@RequestBody Patient p) {
```

```

        return ResponseEntity.status(HttpStatus.CREATED)
            .body(repo.save(p));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Patient> update(@PathVariable String id,
                                          @RequestBody Patient p) {
        return repo.findById(id).map(existing -> {
            p.setId(id);
            return ResponseEntity.ok(repo.save(p));
        }).orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable String id) {
        repo.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}

```

4.3 ConsultationController — Endpoints

Méthode	URL	Statut	Description
GET	/api/consultations	200 OK	Toutes les consultations
GET	/api/consultations/{id}	200 / 404	Consultation par ID
GET	/api/consultations/patient/{patientId}	200 OK	Historique d'un patient
GET	/api/consultations/medecin/{medecinId}	200 OK	Agenda d'un médecin
GET	/api/consultations/statut/{statut}	200 OK	Filtrer par statut
POST	/api/consultations	201 Created	Créer une consultation
PUT	/api/consultations/{id}	200 OK	Modifier / Clôturer
DELETE	/api/consultations/{id}	204	Annuler une consultation

V. Exécution de l'application

Avant de démarrer l'application, MongoDB doit être en cours d'exécution localement. L'application est ensuite lancée via Maven Wrapper :

```

# 1. Démarrer MongoDB (Linux/macOS)
sudo systemctl start mongod

# 2. Démarrer l'application Spring Boot
.\mvnw spring-boot:run

# Confirmation dans la console :
Started HospitalapiApplication in 3.241 seconds (JVM running for 3.891)
Connected to MongoDB: localhost:27017/hospital_db

```

Le serveur écoute sur `http://localhost:8080`. Spring Data MongoDB crée automatiquement les collections `patients`, `medecins` et `consultations` au premier insert.

VI. Validation des services REST via Postman

Dix requêtes ont été exécutées via Postman pour valider l'ensemble des endpoints. Les tests couvrent les quatre méthodes HTTP et les cas d'erreur (404 Not Found).

T1 — Ajouter un patient (POST)

```
POST http://localhost:8080/api/patients
Content-Type: application/json

{
  "nom": "OBAME",
  "prenom": "Marie",
  "dateNaissance": "1985-03-12",
  "telephone": "+241077123456",
  "email": "marie.obame@email.com",
  "groupe_sanguin": "A+",
  "antecedents": {
    "maladies": ["Hypertension"],
    "allergies": ["Pénicilline"],
    "medicaments": ["Amlodipine 5mg"]
  }
}

→ Réponse 201 Created : document créé avec _id MongoDB généré automatiquement
```

T2 — Ajouter un médecin (POST)

```
POST http://localhost:8080/api/medecins
Content-Type: application/json

{
  "nom": "NZAMBA",
  "prenom": "Pierre",
  "specialite": "Cardiologie",
  "telephone": "+241074000001",
  "email": "p.nzamba@hospital.ga",
  "numeroOrdre": "MED-GAB-2019-0042",
  "disponible": true
}

→ Réponse 201 Created
```

T3 — Créer une consultation (POST)

```
POST http://localhost:8080/api/consultations
Content-Type: application/json

{
  "patientId": "665f1a2b3c4d5e6f7a8b9c0d",
```



```
"medecinId": "665f1a2b3c4d5e6f7a8b9c1e",
"date": "2026-06-29T09:30:00",
"motif": "Douleurs thoraciques",
"diagnostic": "Hypertension artérielle grade 2",
"ordonnance": {
  "medicaments": ["Amlodipine 10mg", "Bisoprolol 5mg"],
  "instructions": "1 comprimé matin et soir pendant 30 jours",
  "dureeJours": 30
},
"statut": "terminée"
}

→ Réponse 201 Created
```

T4 — Lister tous les patients (GET)

```
GET http://localhost:8080/api/patients

→ Réponse 200 OK : tableau JSON de tous les patients
```

T5 — Historique d'un patient (GET)

```
GET http://localhost:8080/api/consultations/patient/665f1a2b3c4d5e6f7a8b9c0d

→ Réponse 200 OK : liste de toutes les consultations du patient
```

T6 — Médecins disponibles par spécialité (GET)

```
GET http://localhost:8080/api/medecins/specialite/Cardiologie

→ Réponse 200 OK : liste des cardiologues enregistrés
```

T7 — Modifier le statut d'une consultation (PUT)

```
PUT http://localhost:8080/api/consultations/665f1a2b3c4d5e6f7a8b9c2f
Content-Type: application/json

{
  "statut": "annulée"
}

→ Réponse 200 OK : document mis à jour
```

T8 — Supprimer un patient (DELETE)

```
DELETE http://localhost:8080/api/patients/665f1a2b3c4d5e6f7a8b9c0d
```

→ Réponse 204 No Content : patient supprimé

T9 — Patient introuvable (GET — cas d'erreur)

```
GET http://localhost:8080/api/patients/000000000000000000000000
```

```
→ Réponse 404 Not Found : ResponseEntity.notFound().build()
```

T10 — Consultations par statut (GET)

```
GET http://localhost:8080/api/consultations/statut/planifiée
```

→ Réponse 200 OK : consultations à venir non encore réalisées

VII. Synthèse des endpoints

Test	Méthode	Endpoint	Statut	Opération
T1	POST	/api/patients	201	Ajout patient
T2	POST	/api/medecins	201	Ajout médecin
T3	POST	/api/consultations	201	Création consultation
T4	GET	/api/patients	200	Liste patients
T5	GET	/api/consultations/patient/{id}	200	Historique patient
T6	GET	/api/medecins/specialite/{s}	200	Filtre spécialité
T7	PUT	/api/consultations/{id}	200	Mise à jour
T8	DELETE	/api/patients/{id}	204	Suppression
T9	GET	/api/patients/{id_invalide}	404	Erreur not found
T10	GET	/api/consultations/statut/{statut}	200	Filtre statut

Conclusion

Ce travail pratique a permis de concevoir une API REST hospitalière complète en combinant Spring Boot 3 et Spring Data MongoDB. La gestion des patients, des médecins et des consultations a été implémentée avec les quatre méthodes HTTP, des endpoints de recherche avancée et une gestion correcte des codes de retour HTTP.

L'intégration de MongoDB apporte une flexibilité précieuse : le schéma des documents peut évoluer sans migration de base de données, et l'embedding des antécédents et des ordonnances optimise les performances de lecture en évitant les jointures.

Annotations clés utilisées

Annotation	Rôle
<code>@Document</code>	Mappe une classe Java sur une collection MongoDB
<code>@Id</code>	Désigne le champ identifiant (ObjectId)
<code>@RestController</code>	Expose la classe comme contrôleur REST (JSON)
<code>@RequestMapping</code>	Définit le préfixe de route du contrôleur
<code>@GetMapping</code> / <code>@PostMapping</code>	Associe une méthode à une route HTTP
<code>@PutMapping</code> / <code>@DeleteMapping</code>	Mise à jour et suppression de ressources
<code>@PathVariable</code>	Extrait un paramètre depuis l'URL
<code>@RequestBody</code>	Déséréalise le corps JSON en objet Java
<code>MongoRepository</code>	Interface Spring Data fournissant les opérations CRUD

Évolutions possibles

- Sécurisation des endpoints avec Spring Security + JWT
- Validation des données avec `@Valid` et Bean Validation (`@NotNull`, `@Size`)
- Documentation automatique avec Swagger / OpenAPI 3.0
- Gestion centralisée des exceptions avec `@ControllerAdvice` + `@ExceptionHandler`
- Pagination et tri des résultats avec `Pageable`
- Déploiement avec Docker (Dockerfile + docker-compose pour MongoDB)
- Tests unitaires et d'intégration avec JUnit 5, Mockito et Flapdoodle Embedded MongoDB

La combinaison Spring Boot + MongoDB est l'une des plus populaires dans le développement backend moderne, notamment pour les applications de santé, les plateformes SaaS et les systèmes à fort volume de données.

Réponses aux questions

Question	Réponse
Rôle de @Document ?	Indique à Spring Data MongoDB que la classe est un document persisté. La propriété collection précise le nom de la collection cible dans MongoDB.
Qu'est-ce que MongoRepository ?	Interface Spring Data qui fournit automatiquement les opérations CRUD (save, findById, findAll, deleteById) sans écrire de code SQL ou de requête MongoDB manuellement.
Différence entre save() et update ?	save() insère si le document est nouveau (_id absent) et remplace entièrement s'il existe. updateOne() de MongoDB permet une mise à jour partielle avec \$set.
Pourquoi MongoDB pour une API hospitalière ?	La flexibilité du schéma permet d'ajouter des champs (antécédents, allergies) sans migration. L'embedding des ordonnances améliore les performances de lecture.
Qu'est-ce que l'embedding ?	Technique consistant à stocker un document dans un autre (ex : ordonnance dans consultation). Évite les jointures et améliore les performances quand les données sont toujours consultées ensemble.
Différence ObjectId et String id ?	MongoDB génère un ObjectId (12 octets, horodaté). Spring Data MongoDB le mappe automatiquement en String Java via @Id. L'application n'a pas à gérer la génération d'identifiants.
Rôle de ResponseEntity ?	Permet de contrôler précisément le code de statut HTTP retourné (201 Created, 404 Not Found, 204 No Content), contrairement au retour direct d'un objet qui renvoie toujours 200.
Comment Spring génère-t-il les requêtes ?	Spring Data MongoDB analyse le nom des méthodes de l'interface Repository (findByNom, findBySpecialite) et génère automatiquement la requête MongoDB correspondante à la compilation.